Depth of Field Rendering with Focus Control

JOUANET Camille
Télécom Paris

Contents

1 Introduction

This project is an implementation of a physics-based lens simulation capable of accurate depth of field rendering. This subject is inspired from Lee et al's[5] work Real-Time Lens Blur Effects and Focus Control, which introduced a new GPU-based pipeline to simulate lenses accurately, including focus control, optical aberrations. My method reproduced some of their results following parts of their conceptual method, featuring personal modifications and steps to achieve credible outputs.

2 Implementation

2.1 Lens Modelization

The first thing to do is modelize a lens. The original paper treated both thin and thick lens cases, so I decided to first modelize a thin lens and keep thick lenses as a possible later extension.

As Kolb et al.[2] described, a thin-lens camera model does not require explicit refraction computation using Snell's formula. Instead, refraction is modeled geometrically by enforcing that all rays contributing to a given pixel converge toward the same focal point.

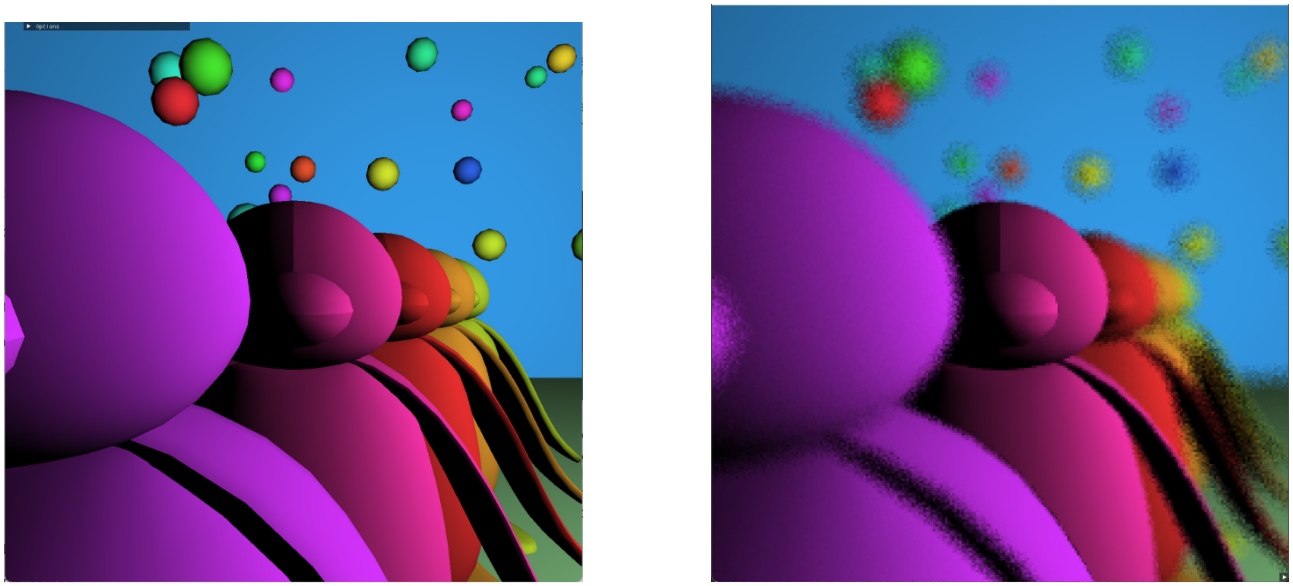
Starting with the easiest case: the pinhole image, which is the image captured with the smallest aperture possible. This condition means light rays will necessarily go through the center of the lens, and thus not be refracted. When arriving on the film (or a screen in our case), the light from a given object point will converge wherever the film is, meaning it will capture an overall sharp image. To reproduce this, each pixel of the screen will trace a ray toward the focal center, which is the camera center in this case, and return the color of the intersected mesh.

To simulate depth of field, the model plays with two parameters: aperture and focal distance. In reality, a ray coming from a given point on the scene will necessarily fill two conditions: going through the lens and converging through its focal point. Reversely, in the model, a ray assigned to a given pixel p will also respect these conditions.

The generated ray position is sampled from a random point on the aperture and directed toward f_p, computed as the intersection between the pinhole ray and the focus plane. If the object lies on the focal plane, all rays converge on it and produce a sharp image. The final pixel color is obtained by averaging all sampled rays.

Figure 1 shows the result of the described process with 16 accumulated samples, selected randomly following a uniform law.

These initial results allowed me to validate my algorithm and architecture, but it is immediately apparent that the result is noisy and lacks realism.



(a) (b)

Figure 1: (a) Original image (b) rendered image with uniform sampling

2.2 Improve Results

2.2.1 Sampling disk method

The sampling functions can be found in the Helper/Utils.h file.

Uniform

As mentioned above, the first sample method I used was uniform distribution. I followed a naive approach: sampling enough random points on a disk should give a significant average. However, considering this sampling will be scattered uniformly based on concentric circles inside the disk (see Figure 2), this sample does not seem appropriate anymore. This concentration brings noise to the blur which feels unnatural to the human eye and therefore should be decreased as much as possible.

Stratified

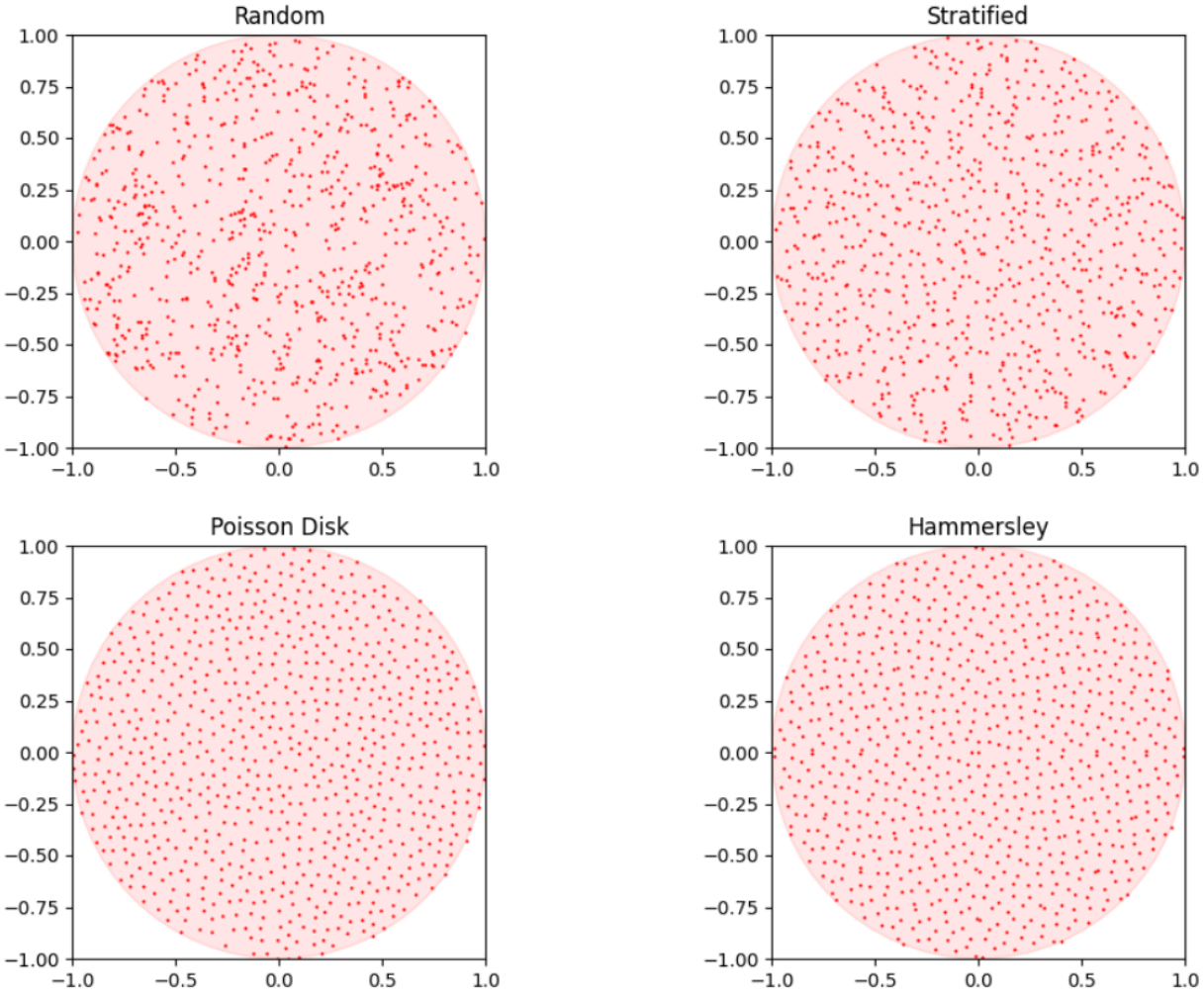
To solve this distribution issue, I decided to proceed with a stratified approach: if the sample concentration per area is not regular, I can enforce it. This stratified method divides the disk into a grid containing N cells, where N is the number of samples per pixel. The function is called passing the current sample N_i in argument, to sample uniformly in the N_i^{th} cell. The result feels better, but still noisy: the blurred objects have a "still-loading" appearance that feels unnatural.

Poisson

A Poisson disk sampling approach was also implemented to enforce a minimum distance between samples. However, no significant visual improvement was observed, which may be due to the relatively low number of samples used per pixel. Moreover, although Poisson disk improves distribution uniformity over the disk, it still takes a significant amount of samples to show acceptable results. The challenge is thus to improve the find a sampling technique which allows for early convincing results.

Hammersley This idea led me to shift strategy and opt for a quasi–random sampling using a Hammersley sequence[6]. Unlike purely random sampling, Hammersley generates a deterministic low-discrepancy sequence that progressively covers the domain in a highly uniform manner. Like Poisson sampling, Hammersley sampling distributes points more evenly over the lens, which reduces structured noise and improves visual stability early in the accumulation process.

This method shows smoother noise, visually high results, and has the advantage of a faster perceptual convergence, needing only a few passes to generate decent blur zones.



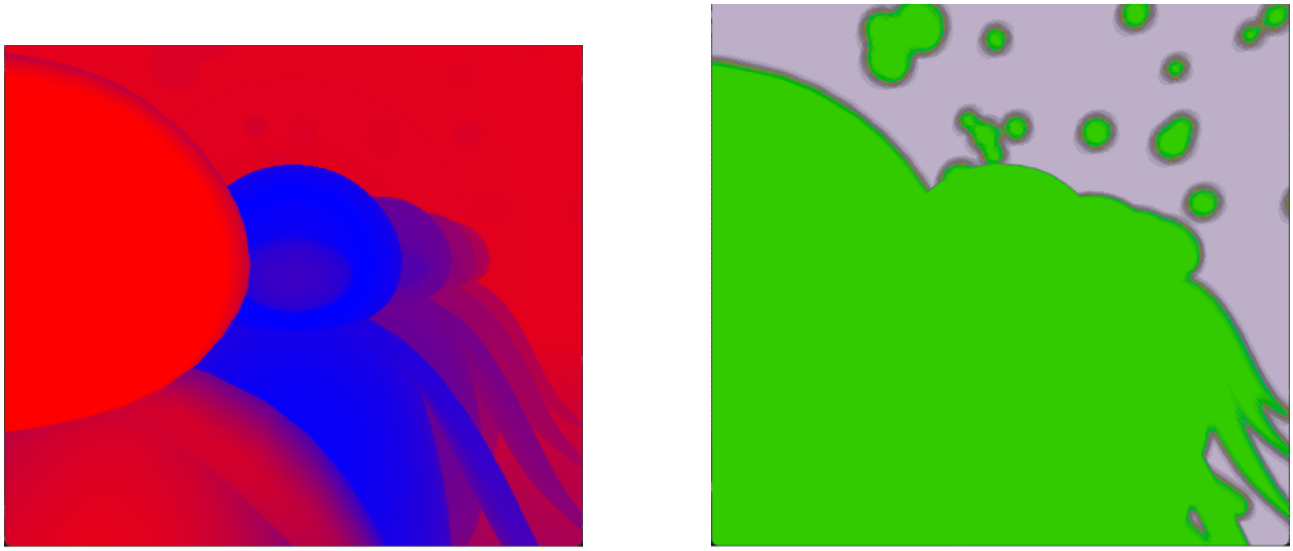
(a) (b)

Figure 2: Comparison of different sampling on a disk (N=800)

2.2.2 Multiple Hit Consideration

Inspired by N-buffer approaches and depth peeling techniques presented in Lee et al's paper, I tried considering multiple hits per ray when computing depth of field. The idea was to reduce occlusion produced by out-of-focus foreground elements on the sharp meshes behind. The first two hits of each ray are collected along their path. If the circle of confusion (CoC) of the first hit falls above a given threshold, the next hit adds a small contribution to the result.

However, as this attenuation is purely heuristic and not physically based, the resulting improvements are limited and introduced several artifacts that would need neighbor check to be attenuated. For these reasons, while multi-layer accumulation based on CoC was conceptually appealing, it proved difficult to manipulate heuristically without further weighting my algorithm. Which is why I ended up giving up this idea. (fig of coc and layerhit)



(a) (b)

Figure 3: (a) Circle of confusion visualization [blue for low, red for high] (b) area with at lest 2 layers (right)

2.3 Optimization

A major limitation of my project, which is not directly conveyed by the figures presented in this report, is the computational cost of my depth of field rendering.

One of the key contributions of Lee et al.'s work lies in the introduction of N-Buffers[3], a set of modified depth buffers designed to reduce the cost of ray-scene intersection by restricting ray traversal to a limited number of depth layers per pixel. Meaning rays intersect several 2D layers packed with data rather than a full 3D scene. These buffers are constructed using a depth peeling strategy[4] and allow the lens simulation to operate efficiently within a real-time GPU pipeline.

Although I implemented a depth peeling algorithm to extract multiple depth layers per pixel (see fig), I did not manage to integrate the data into my pipeline in the same manner as the original paper approach. As a consequence, my implementation retains a full CPU-based ray tracing pipeline, where rays are still tested against the entire scene geometry.

While the original paper achieves real-

time lens simulation, my implementation as it stood in its first iteration required several minutes to accumulate enough samples to produce visually convincing results.

A significant part of the work was thus aimed at optimizing the ray tracing stage the best I could to reduce rendering time.

2.3.1 Intersection Tests

As is often the case with ray tracing, the primary source of complexity comes from ray/triangle intersection tests. My first version was using a brute force test that checked whether a ray intersected the bounding sphere of a mesh and then tested all the triangles in said-mesh if needed. This method is obviously very resource-intensive and was used for ease of implementation in order to obtain the first visible results. As the project progressed, I first improved the initial intersection test by calculating the AABB boxes of the scene's meshes, and eventually implemented a Bounding Volumes Hierarchy, which tremendously accelerated the rendering.

2.3.2 Early Exit

To reduce unnecessary sampling, I explored an early exit strategy to detect pixel convergence during progressive rendering. Since some pixels hardly change after the first few samples, the algorithm should avoid unprofitable calculations. I first thought about a naïve approach which would consist in measuring the variance of RGB values across samples and stopping once a predefined threshold is reached.

However, this criteria would have proven unreliable in this context of depth-of-field rendering. Since the algorithm introduces color mixing regardless of depth layers to create blur, high RGB variations will happen even when the resulting blur is already perceptually stable.

Since the goal of depth-of-field is to produce visually convincing blur rather than numerically stable color values, I needed to find a perceptual noise criteria. For this reason, pixel convergence was instead evaluated using luminance variance, computed as a weighted sum of RGB components following the sRGB/Rec.709 standard[1].

Luminance better reflects the sensitivity of the human visual system, taking into account the high perceptive response to green compared to red and blue for example. This provides a more reliable indicator of perceptual noise.

$$Luminance = R*0.2126+G*0.7152+B*0.0722 \quad (1)$$

In practice, although this approach provided better computation time, even with low threshold, artifacts would appear in blurred areas, which made me forgo this method. (see Fig?)

2.3.3 Parallelization

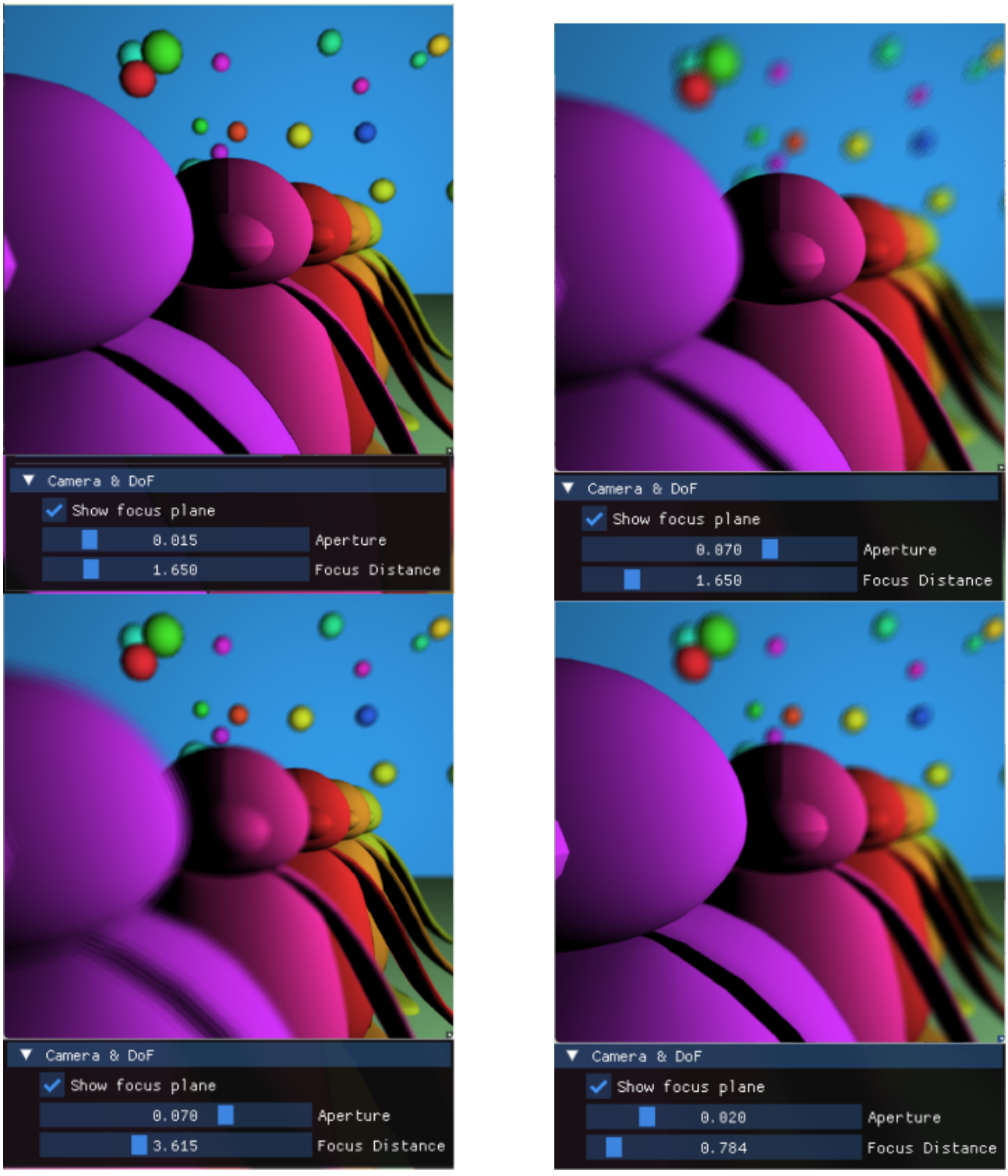
In ray tracing, each pixel can be computed independently, which makes the workload well-suited for multi-threading.

The rendering process is thus parallelized using a thread pool. Small blocks of pixels are distributed across multiple worker threads, which independently:

1. generates lens rays for a subset of pixels
2. traces them through the BVH
3. updates the pixel's accumulated color

Due to the aforementioned pixel convergence testing, designing this multi-thread raised challenges, as converged pixels had to be dynamically removed from the active set. This required additional synchronization mechanisms and reduced the simplicity of the parallel pipeline.

3 Results



(a) (b)

Figure 4: Outputs with different model parameters

4 Discussion

As it is, my project is missing the most important part of Lee et al's work, which makes it incomplete and slow to produce convincing results. The next course of action would be to include N-Buffers using depth peeling before generation rays as done already in this method. I blame my poor prioritization of tasks for this missing aspect, which led me to overthink about optimizing the CPU performances rather than implementing the original pipeline, as imperfect it may be, before. The next steps for this project would then naturally be to make use of the depth peeling algorithm to work with N-Buffers and refactor the rendering pipeline, which would greatly improve both rendering time and visual result by layering the 3D scene.

For these reasons, my implementation, focused on CPU usage, is logically way slower than the original paper ones, which created a GPU-based tracing method. Still, I managed to get convincing blur results following the same physics-based lens they used, which can be controlled by model parameters.

Another feature that could be introduced would be the multiple depth of field handling presented in their paper. This would allow for artistic effects impossible to recreate in real life and add some user interaction, which could complement the click-based depth of field updater present in my project.

References

[1] What is the efficient way to calculate human eye contrast difference for rgb values? https://stackoverflow.com/questions/56198778/what-is-the-efficient-way-to-calculate-human-eye-contrast-difference-for-rgb-val/5620073856200738.

[2] Don Mitchell Craig Kolb and Pat Hanrahan. A realistic camera model for computer graphics. In *In Proceedings of the 22nd annual conference on Computer graphics and interactive techniques (SIGGRAPH '95).*, page 317–324, 1995.

[3] Xavier Décoret. N-buffers for efficient depth map query. *Computer Graphics Forum*, 24(3):393–400, 2005.

[4] Cass W. Everitt. Interactive order-independent transparency. 2001.

[5] Elmar Eisemann Sungkil Lee and Hans-Peter Seidel. Real-time lens blur effects and focus control. *ACM Trans.*, Graph. 29(4):7, 2010.

[6] Tien-Tsin Wong, Wai-Shing Luk, and Pheng-Ann Heng. Sampling with hammersley and halton points. *Journal of Graphics Tools*, 2, 01 1997.